Zero to Hero Study Handbook: Production-Grade AI Task Planning & Productivity Agent

This handbook is a source-grounded guide to this repository. Every section maps to real files and real code objects in the project.

Module 1: Foundations & Architecture

What this project does

This project implements a local-first AI productivity assistant that converts messy task input into structured plans, schedules those tasks, stores history, and exposes the system through API, CLI, and Streamlit UI.

Core delivered capabilities are implemented in:

- Multi-stage planning workflow with LangGraph: `src/task_planning_agent/agent/graph.py`, `src/task_planning_agent/agent/nodes.py`
- Structured task schema and report schema: `src/task_planning_agent/schemas.py`
- API surface: `src/task_planning_agent/api/routers/core.py`
- UI: `src/task_planning_agent/ui/streamlit_app.py`
- Persistent memory and analytics: `src/task_planning_agent/memory`, `src/task_planning_agent/analytics`

Main use cases supported by code:

- Parse messy lists/notes into tasks (`parse_messy_input`, `TaskExtractor.extract`)
- Prioritize tasks via switchable frameworks (`score_tasks` + strategy classes)
- Build dependency graph and detect cycles (`DependencyPlanner.build_dependencies`)
- Generate schedule blocks (`ScheduleOptimizer.schedule`)
- Persist plans/tasks/preferences (`SQLiteMemoryStore`) and semantic memory (`ChromaSemanticStore`)
- Export calendar schedules as ICS (`CalendarService.export_ics`)
- Replan from most recent history (`PlanningService.replan`)

Core paradigms and patterns used here

Definitions first:

- Stateful workflow graph: sequential node execution over shared state. Here: LangGraph `StateGraph(AgentState)` with explicit edges.
- Strategy pattern: interchangeable algorithms behind one interface. Here: priority strategies implementing `PrioritizationStrategy`.
- Layered architecture: presentation layer -> service/workflow layer -> storage/tool layer.
- Contract-first adapters/stubs: external integrations are defined by interfaces and stubs before live credentials.

How those paradigms appear in this repo:

- Stateful workflow graph:
  - Graph definition: `build_graph()` in `agent/graph.py`
  - Typed state contract: `AgentState` in `agent/state.py`
  - Node logic container: `WorkflowNodes` in `agent/nodes.py`
- Strategy pattern:
  - Base interface: `PrioritizationStrategy` in `prioritization/base.py`
  - Strategy implementations: `prioritization/strategies.py`
  - Runtime selection registry: `STRATEGY_REGISTRY` in `prioritization/registry.py`
- Layered architecture:
  - Entry points: `app.py`, `streamlit_app.py`, CLI script `task-agent` in `pyproject.toml`
  - Service façade: `PlanningService` in `agent/service.py`
  - Storage and analytics layers: SQLite, ChromaDB, DuckDB, MLflow modules
- Contract-first adapters/stubs:
  - Tool base + registry: `tools/base.py`, `tools/registry.py`
  - External connector contract and stubs: `tools/connector_contract.py`, `tools/connectors.py`
  - Google calendar stub: `calendar/adapters/google_stub.py`

Architecture description

Primary component chain:

1. Input normalization and extraction
2. Priority scoring and dependency planning
3. Schedule generation and validation
4. Reflection/recommendation
5. Persistence + analytics logging
6. Report generation for API/CLI/UI consumers

Storage systems by responsibility:

- SQLite: users, preferences, tasks, plans, reflections
- ChromaDB: semantic retrieval over plan/task text
- DuckDB: analytics metrics event store
- MLflow: metrics logging sink (with DuckDB fallback)

Text-based architecture flow:

```
User Input (API / CLI / Streamlit)
        |
        v
[PlanningService.plan()]
        |
        v
[LangGraph StateGraph]
```

```
START
  -> planner
     - TaskExtractor.extract()
     - score_tasks()
     - DependencyPlanner.build_dependencies()
     - optional _llm_refinement()
  -> scheduler
     - ScheduleOptimizer.schedule()
  -> validator
     - issue checks (empty tasks/blocks, high-risk count)
  -> reflection
     - ReflectionAgent.reflect()
     - RecommendationAgent.suggest()
  -> memory
     - MemoryManager.persist_plan()
     - SQLite + Chroma upsert
     - AnalyticsEngine.snapshot() -> MLflow + DuckDB
  -> reporter
     - ReportGenerator.generate()
END
      |
      v
PlanReport returned to caller
```

Interface layer mapping:

- FastAPI app + routes: `src/task_planning_agent/api`
- Typer CLI: `src/task_planning_agent/cli.py`
- Streamlit dashboard: `src/task_planning_agent/ui/streamlit_app.py`

Module 2: Repository Map

Focus these files first when onboarding.

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`pyproject.toml`	Packaging, dependencies, CLI entrypoint, tooling config	`[project]`, `[project.scripts]` `task-agent = task_planning_agent.cli:app`	`requires-python`, `dependencies`, `optional-dependencies.dev`, `pytest`/`ruff`/`mypy` sections

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`configs/config.yaml`	Runtime YAML configuration	N/A (data file)	`paths.sqlite_path`, `paths.chroma_dir`, `llm.enabled`, `llm.families`, `planner.default_priority_strategy`, `scheduling.*`, `api.host/port`, `streamlit.host/port`
`.env.example`	Environment-variable template	N/A (data file)	`TPA_ENV`, `TPA_LOG_LEVEL`, `TPA_OLLAMA_BASE_URL`, `TPA_JWT_SECRET`, connector/token keys
`app.py`	Python entrypoint for API server	`run()` import from `task_planning_agent.api.app`	N/A
`streamlit_app.py`	Python entrypoint for Streamlit	`render()` import from `task_planning_agent.ui.streamlit_app`	N/A
`src/task_planning_agent/config.py`	Config/env-backed runtime settings	`RuntimeSettings`, `AppConfig`, `get_runtime_settings()`, `load_config()`	`env_prefix="TPA_"`, default config path, `configs/config.yaml`
`src/task_planning_agent/schemas.py`	Canonical domain/API schemas	`Task`, `PlanRequest`, `ReplanRequest`, `PlanReport`, `PlanSession`, `ScheduleBlock`, `UserPreference`, enums	`PriorityStrategy`, `RiskLevel`, `TaskStatus` values
`src/task_planning_agent/service.py`	High-level planning/replanning façade	`PlanningService.__init__`, `plan()`, `replan()`	defaults for sqlite/chroma paths, analytics URI, scheduler params
`src/task_planning_agent/graph.py`	LangGraph agent construction	`build_graph()`	Node order and edges START->...->END

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`src/task_planning_agent/agent/state.py`	Agent graph state contract	`AgentState` `TypedDict`	keys: `tasks`, `dependencies`, `schedule_blocks`, `report`, etc.
`src/task_planning_agent/agent/nodes.py`	Node implementations for graph stages	`WorkflowNodes.planner_node`, `_llm_refinement`, `scheduler_node`, `validator`, `reflection_node`, `memory_node`, `reporter_node`	`llm_enabled`, shared dependencies (`memory`, `analytics`, `scheduler`)
`src/task_planning_agent/ingestion/parser.py`	Messy text parsing heuristics	`ParsedTaskCandidate`, `parse_messy_input()`, `normalize_notes_block()`	regex patterns: `TASK_LINE_PATTERN`, `ESTIMATE_PATTERN`, `DEADLINE_HINT_PATTERN`
`src/task_planning_agent/ingestion/extractor.py`	Candidate-to-Task conversion with confidence/risk heuristics	`ExtractionResult`, `TaskExtractor.extract()`, `_candidate_to_task()`	`default_estimate_minutes`
`src/task_planning_agent/prioritization/strategies.py`	Scoring algorithm implementations	`EisenhowerStrategy`, `MoscowStrategy`, `AbcdeStrategy`, `RiceStrategy`, `IceStrategy`, `WsjfStrategy`, `UrgencyImportanceStrategy`, `WeightedStrategy`	helper formulas `_deadline_urgency`, `_effort_penalty`
`src/task_planning_agent/prioritization/registry.py`	Strategy registry and sorting	`STRATEGY_REGISTRY`, `score_tasks()`	selected `PriorityStrategy` enum key
`src/task_planning_agent/dependencies/planner.py`	Dependency DAG build + cycle detection	`DependencyPlanner.build_dependencies()`	`allow_unknown_task_ids` checks and `topo_rank` annotation
`src/task_planning_agent/scheduling/optimizer.py`	Time-block scheduling	`ScheduleOptimizer.schedule()`	`workday_start`, `workday_end`, `break_minutes`, `deep_work_block_minutes`

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
src/task_planning_agent/memory/sqlite_store.py	SQLite persistence	SQLiteMemoryStore methods (upsert_user, save_tasks, save_plan, etc.)	SQL tables: users, preferences, tasks, plans, reflections
src/task_planning_agent/memory/chroma_store.py	Chroma memory persistence/query	ChromaSemanticStore.upsert(), .query()	collection_name, _naive_embedding()
src/task_planning_agent/memory/manager.py	Orchestrates SQLite + Chroma	MemoryManager.persist_plan(), history(), semantic_search(), search_tasks()	sqlite_path, chroma_path, default "plan_memory"
src/task_planning_agent/analytics/analytics.py	Agent analytics computation + logging	AnalyticsEngine.snapshot(), _log()	mlflow_tracking_uri, metrics fields in AnalyticsSnapshot
src/task_planning_agent/analytics/duckdb_store.py	Analytics storage/query	DuckDBStore.log_metric(), fetch_weekly_summary()	metric, analytics_events
src/task_planning_agent/reports/generator.py	User-facing report construction	ReportGenerator.generate()	summary_text and PlanReport mapping
src/task_planning_agent/calendar/service.py	Calendar orchestration	CalendarService.import_ics(), export_ics(), detect_conflicts(), available_slots()	ICS data, Google stub usage
src/task_planning_agent/api/app.py	FastAPI app factory and server run	create_app(), run()	API title/version, host/port from config
src/task_planning_agent/api/deps.py	Dependency injection and auth identity	get_service(), get_current_user()	HTTPBearer, JWT secret/algorithm loading, caches

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`src/task_planning_agent/api/routers/core.py`	REST endpoints for auth, planning, tasks, history, search, calendar, preferences, reporting, health, strategies, and tools	`/auth/register`, `/auth/login`, `/plan`, `/replan`, `/tasks`, `/history`, `/search`, `/calendar`, `/calendar/export`, `/preferences`, `/report`, `/health`, `/strategies`, `/tools/{tool_name}`	RBAC check (`current_user`) and payload models
`src/task_planning_agent/cli.py`	CLI entrypoints for plan/replan/search/report/serve workflows	`plan_cmd`, `replan_cmd`, `search_cmd`, `report_cmd`, `serve_api`, `serve_ui`, `verify`	default timezone, output path, Streamlit host/port
`src/task_planning_agent/ui/streamlit_app.py`	Dashboard rendering and interactions	`render()`, `dashboard_page()`, `weekly_page()`, `calendar_page()`, etc.	session keys like `latest_report`, `_autoplan_done`
`tests/`	Behavioral and contract tests	`test_agent_workflow`, `test_api`, `test_memory`, `test_calendar`, `test_dependency_planner`, `test_scheduler`, `test_parser`, `test_prioritization`	Assertions define expected behavior surfaces

Module 3: Core Execution Flows

Flow A: Main planning flow (`POST /plan -> PlanReport`)

This is the primary runtime path.

Step-by-step:

1. Request enters FastAPI router function `plan()` in `api/routers/core.py`.

2. Router enforces ownership/RBAC: payload `user_id` must equal token subject unless role is `admin`.
3. Router calls `PlanningService.plan(...)` in `agent/service.py`.
4. Service invokes graph with initial state keys: `user_id`, `raw_input`, `strategy`, `timezone`.
5. Node `planner`:
   - `TaskExtractor.extract(raw_input)`
   - `score_tasks(extraction.tasks, strategy)`
   - `DependencyPlanner.build_dependencies(tasks)`
   - optional `_llm_refinement(...)` if `llm.enabled` is true.
6. Node `scheduler`:
   - `ScheduleOptimizer.schedule(tasks, timezone)`
   - marks matching tasks as `TaskStatus.SCHEDULED`.
7. Node `validator`:
   - appends issue strings for empty extraction, empty schedule, high-risk blocks.
8. Node `reflection`:
   - `ReflectionAgent.reflect(plan_id, blocks)`
   - `RecommendationAgent.suggest(blocks)`
9. Node `memory`:
   - create `PlanSession`
   - `MemoryManager.persist_plan(session)` -> SQLite + ChromaDB
   - `AnalyticsEngine.snapshot(...)` -> MLflow + DuckDB
10. Node `reporter`:
    - `ReportGenerator.generate(session)` -> `PlanReport`
11. Service returns `output["report"]` to API.

Input shape (`PlanRequest` from `schemas.py`):

```
{
  "user_id": "string",
  "raw_input": "string",
  "strategy": "eisenhower|moscow|abcde|rice|ice|wsjf|urgency_importance|weighted",
  "timezone": "string"
}
```

Output shape (`PlanReport` and nested `ScheduleResponse`):

```
{
  "plan_id": "string",
  "user_id": "string",
  "generated_at": "datetime",
  "summary": "string",
  "schedule": [
    {
      "task": "string",
      "priority": 0.0,
      "deadline": "datetime|null",
```

```
      "estimated_duration": 0,
      "dependencies": ["task_id"],
      "suggested_start_time": "datetime",
      "suggested_end_time": "datetime",
      "confidence": 0.0,
      "reasoning": "string",
      "risk_level": "low|medium|high"
    }
  ],
  "reflections": ["string"],
  "recommendations": [
    {"category": "string", "suggestion": "string", "impact": "string"}
  ],
  "analytics": {
    "completed_tasks": 0,
    "completion_rate": 0.0,
    "average_delay_minutes": 0.0,
    "planning_accuracy": 0.0,
    "focus_time_minutes": 0,
    "deep_work_minutes": 0,
    "meetings_minutes": 0,
    "context_switches": 0,
    "energy_score": 0.0,
    "burnout_score": 0.0,
    "weekly_productivity_score": 0.0
  }
}
```

Flow B: Replanning (`POST /replan`)

Implemented in `PlanningService.replan(...)`:

- Pull latest session via `self.memory.history(user_id, limit=1)`.
- If no history exists, fallback to a normal `plan(...)` with synthesized input text.
- If history exists, merge:
  - previous raw input
  - new `reason`
  - `additional_input`
- Re-run `plan(...)` with strategy fixed to `PriorityStrategy.WSJF` and timezone `"Asia/Kolkata"`.

`ReplanRequest` input shape:

```
{
  "user_id": "string",
  "reason": "string",
```

```
  "additional_input": "string"
}
```

Flow C: Parsing -> extraction -> prioritization internals

1. `parse_messy_input(raw_input)` in `ingestion/parser.py`:
   - Splits non-empty lines.
   - Handles bullet or numbered lines using `TASK_LINE_PATTERN`.
   - Extracts estimates (`ESTIMATE_PATTERN`), people (`@name`), project (`#project`), and deadline hints.
2. `TaskExtractor._candidate_to_task(...)` in `extraction/extractor.py`:
   - Converts parser candidate into `Task`.
   - Sets `deadline`, `estimated_minutes`, `people`, `project`.
   - Builds heuristic `confidence` and initial `risk_level`.
3. `score_tasks(tasks, strategy)` in `prioritization/registry.py`:
   - Uses `STRATEGY_REGISTRY[strategy]`.
   - Writes `task.priority_score` and appends rationale to `task.reasoning`.
   - Returns tasks sorted descending by score.

Parser candidate data shape (`ParsedTaskCandidate`):

```
raw_line: str
title: str
description: str
deadline_text: str | None
estimated_minutes: int | None
people: list[str]
project: str | None
```

Flow D: Dependency and scheduling flow

Dependency phase:

- `DependencyPlanner.build_dependencies(tasks)` creates a `networkx.DiGraph`.
- Adds edges from each declared dependency ID to child task ID.
- If graph is not DAG:
  - computes cycles with `nx.simple_cycles`
  - emits issue messages.
- For DAGs, computes topological order and appends `topo_rank` token to each task's reasoning.

Scheduling phase:

- `ScheduleOptimizer.schedule(tasks, timezone)`:
  - starts cursor at configured `workday_start`.
  - iterates tasks by descending `priority_score`.
  - computes block duration from `estimated_minutes` (minimum 15).
  - advances to next day when past `workday_end`.
  - marks task high risk when projected end exceeds deadline.

  - creates `ScheduleBlock` records and inserts configured breaks.

`ScheduleBlock` shape:

```
task_id: str
task_name: str
suggested_start_time: datetime
suggested_end_time: datetime
priority: float
confidence: float
reasoning: str
risk_level: RiskLevel
```

Flow E: Persistence and semantic memory

Persistence path is centralized in `MemoryManager.persist_plan(session)`:

- `sqlite.save_tasks(user_id, tasks)`
- `sqlite.save_plan(session)`
- `chroma.upsert(...)` for each task text and one plan text record

SQLite schema initialization happens automatically in `SQLiteMemoryStore._init_db()` on store creation. Tables:

- `users`
- `preferences`
- `tasks`
- `plans`
- `reflections`

Semantic search path:

- `MemoryManager.semantic_search(query)` -> `ChromaSemanticStore.query(query_text, n_results)`
- Query returns list of dictionaries with keys:
  - `document`
  - `metadata`
  - `distance`

Flow F: API supporting flows (`/health`, `/search`, `/calendar/export`)

`GET /health`:

- collects runtime metrics via `RuntimeMonitor.collect()`.
- returns:
  - `status`
  - `time`
  - `tools` (from `ToolRegistry.list_tools()`)
  - `runtime` (`RuntimeMetrics` fields)
  - `db` (SQLite path)

`GET /search`:

- returns both lexical and semantic results:

```
{
  "tasks": [ { "...task fields..." } ],
  "semantic": [ { "document": "...", "metadata": {...}, "distance": 0.0 } ]
}
```

`POST /calendar/export`:

- retrieves latest session (`history(..., limit=1)`)
- exports `schedule_blocks` through `CalendarService.export_ics(output_path, blocks)`
- returns `{ "path": "..." }`

Flow G: UI and CLI runtime surfaces

CLI:

- Entry script: `task-agent` -> `task_planning_agent.cli:app`
- Core commands:
  - `plan`
  - `replan`
  - `search`
  - `report`
  - `serve-api`
  - `serve-ui`
  - `verify`

Streamlit:

- `render()` sets navigation pages:
  - Dashboard
  - Today's Plan
  - Weekly Planner
  - Task Inbox
  - Calendar
  - Memory
  - Analytics
  - Reports
  - Settings
- `dashboard_page()` can auto-generate an initial plan once per session (`_autoplan_done` guard).

Module 4: Setup & Run Guide

This section documents how the repository is intended to be installed and run on a clean machine.

1. Prerequisites

- OS: Linux (project context targets Ubuntu)
- Python: 3.12 (from `pyproject.toml`)
- Package manager: `uv`
- Optional local inference backend: Ollama running at `http://localhost:11434` if you enable `llm.enabled: true`

2. Install dependencies

```
uv venv --python 3.12 .venv
source .venv/bin/activate
uv sync --extra dev
```

3. Environment configuration

Create `.env` from `.env.example`:

```
cp .env.example .env
```

Required runtime keys (minimum):

- `TPA_ENV`
- `TPA_LOG_LEVEL`
- `TPA_OLLAMA_BASE_URL`
- `TPA_JWT_SECRET`

Optional integration keys:

- Calendar/connectors:
  - `TPA_GOOGLE_CALENDAR_CLIENT_ID`
  - `TPA_GOOGLE_CALENDAR_CLIENT_SECRET`
  - `TPA_GOOGLE_CALENDAR_REFRESH_TOKEN`
  - `TPA_JIRA_TOKEN`
  - `TPA_NOTION_TOKEN`
  - `TPA_TODOIST_TOKEN`
  - `TPA_SLACK_BOT_TOKEN`
  - `TPA_EMAIL_IMAP_PASSWORD`
  - `TPA_WHATSAPP_TOKEN`
- Optional tool providers:
  - `TPA_WEATHER_API_KEY`
  - `TPA_WEB_SEARCH_API_KEY`

4. Main configuration file

Edit `configs/config.yaml` to change behavior without code edits:

- LLM:
  - `llm.enabled`
  - `llm.default_family`

  - `llm.families`
- Planning/scheduling:
  - `planner.default_priority_strategy`
  - `scheduling.workday_start`
  - `scheduling.workday_end`
  - `scheduling.default_break_minutes`
- Persistence:
  - `paths.sqlite_path`
  - `paths.chroma_dir`
  - `memory.chroma_collection`
- Serving:
  - `api.host`, `api.port`
  - `streamlit.host`, `streamlit.port`

5. Typical command sequences

Run API:

```
uv run python app.py
```

Run Streamlit:

```
uv run streamlit run streamlit_app.py
```

Run CLI examples:

```
uv run task-agent plan --user-id ahmad --input-text "- Finish report by tomorrow 5pm 90min"
uv run task-agent replan --user-id ahmad --reason "Urgent blocker" --additional-input "- Fi
uv run task-agent search --user-id ahmad --query report
uv run task-agent report --user-id ahmad --output artifacts/reports/latest.json
```

Run notebook executor script:

```
uv run python scripts/run_notebook.py
```

Generate visualization/report artifacts script:

```
uv run python scripts/generate_artifacts.py
```

6. Database migration/seeding notes

There are no separate migration scripts in this repository. Initialization is automatic:

- SQLite tables are created in `SQLiteMemoryStore._init_db()` when the store is constructed.
- DuckDB table `analytics_events` is created in `DuckDBStore._init_tables()`.
- ChromaDB collection is created/retrieved in `ChromaSemanticStore.__init__()`.

7. Handbook PDF export

This markdown file can be exported directly with Pandoc:

```
pandoc docs/Zero_to_Hero_Study_Handbook.md --pdf-engine=xelatex -o docs/Zero_to_Hero_Study_H
```

Module 5: Study Plan & Practice Exercises

Ordered study plan

1. Start with data contracts and config:
   - `src/task_planning_agent/schemas.py`
   - `configs/config.yaml`
   - `src/task_planning_agent/config.py`
2. Understand the planning pipeline:
   - `src/task_planning_agent/agent/state.py`
   - `src/task_planning_agent/agent/graph.py`
   - `src/task_planning_agent/agent/nodes.py`
   - `src/task_planning_agent/agent/service.py`
3. Understand extraction and scoring internals:
   - `src/task_planning_agent/ingestion/parser.py`
   - `src/task_planning_agent/extraction/extractor.py`
   - `src/task_planning_agent/prioritization/*`
   - `src/task_planning_agent/dependencies/planner.py`
   - `src/task_planning_agent/scheduling/optimizer.py`
4. Understand persistence and analytics:
   - `src/task_planning_agent/memory/*`
   - `src/task_planning_agent/analytics/*`
   - `src/task_planning_agent/reports/generator.py`
5. Understand interfaces:
   - `src/task_planning_agent/api/*`
   - `src/task_planning_agent/cli.py`
   - `src/task_planning_agent/ui/streamlit_app.py`
6. Validate understanding with tests:
   - `tests/test_parser.py`
   - `tests/test_prioritization.py`
   - `tests/test_dependency_planner.py`
   - `tests/test_scheduler.py`
   - `tests/test_memory.py`
   - `tests/test_api.py`
   - `tests/test_agent_workflow.py`
   - `tests/test_calendar.py`

Practice exercises

1. Exercise: Trace how a single bullet task becomes a scheduled block.
   - Read: `parse_messy_input()`, `TaskExtractor.extract()`,

`score_tasks()`, `ScheduleOptimizer.schedule()`.

2. Exercise: Explain where and how `priority_score` is assigned and sorted.
   - Read: `prioritization/registry.py` and `prioritization/strategies.py`.
3. Exercise: Show how cycle detection works for task dependencies.
   - Read: `DependencyPlanner.build_dependencies()` and `tests/test_dependency_planner.py`.
4. Exercise: List all checks that can produce issues in the `validator` node.
   - Read: `WorkflowNodes.validator()`.
5. Exercise: Identify exactly what `/health` returns and where each field comes from.
   - Read: `api/routers/core.py` `health()` and `observability/monitor.py`.
6. Exercise: Explain the difference between lexical task search and semantic search in `/search`.
   - Read: `MemoryManager.search_tasks()`, `MemoryManager.semantic_search()`, `SQLiteMemoryStore.search_tasks_text()`, `ChromaSemanticStore.query()`.
7. Exercise: Explain how `replan` merges prior context.
   - Read: `PlanningService.replan()`.
8. Exercise: Identify every table created in SQLite and what each stores.
   - Read: SQL script in `SQLiteMemoryStore._init_db()`.
9. Exercise: Describe which integrations are real implementations vs stubs.
   - Read: `calendar/adapters/ics.py`, `calendar/adapters/google_stub.py`, `tools/connectors.py`.
10. Exercise: Map each Streamlit page to its function and data dependency.
    - Read: page functions and `pages` dict in `ui/streamlit_app.py`.

Model answers / solution outlines

1. Task-to-block path:
   - `raw_input` lines -> `ParsedTaskCandidate` -> `Task` -> scored/sorted tasks -> `ScheduleBlock` list.
2. Priority assignment:
   - `score_tasks()` selects strategy, calls `strategy.score(task)`, sets `task.priority_score`, sorts descending.
3. Cycle detection:
   - NetworkX `is_directed_acyclic_graph`; if false, `simple_cycles` generates cycle issue messages.
4. Validator issues:
   - No tasks extracted.
   - Tasks exist but no schedule blocks.
   - One or more high-risk blocks detected.
5. `/health` composition:
   - `status`, `time` in route.
   - `tools` from `ToolRegistry.list_tools()`.
   - `runtime` from `RuntimeMonitor.collect()`.
   - `db` from `service.memory.sqlite.db_path`.
6. Search modes:

    - Lexical: case-insensitive JSON-string contains search over stored task payloads.
    - Semantic: Chroma vector query using `_naive_embedding`.
7. Replan merge:
    - If prior session exists, `merged_input` includes previous raw input + reason + additional input; then `plan()` reruns.
8. SQLite tables:
    - `users` auth and role
    - `preferences` serialized `UserPreference`
    - `tasks` serialized `Task`
    - `plans` serialized `PlanSession`
    - `reflections` serialized `ReflectionRecord`
9. Integrations:
    - Live local implementation: ICS import/export.
    - Stub/contract mode: Google calendar and connectors (GitHub/Jira/Notion/Todoist/Google Tasks/Slack/Email/WhatsApp).
10. Streamlit mapping:

- `pages` dict binds labels to page functions; most pages read `st.session_state["latest_report"]` or memory queries.

Understanding Checklist

Use this checklist to self-verify mastery:

- Can you explain the full `plan` execution path from API request to `PlanReport`?
- Can you describe every field in `Task`, `ScheduleBlock`, and `PlanReport` and where they are produced?
- Can you compare at least three prioritization strategies in this codebase and explain formula differences?
- Can you explain how dependency cycle detection is implemented and surfaced as issues?
- Can you explain how scheduling decides start/end times and when risk is escalated?
- Can you explain where persistent data lives (SQLite, ChromaDB, DuckDB) and what each stores?
- Can you explain JWT auth flow (`register`, `login`, `get_current_user`) and RBAC checks in routes?
- Can you explain what parts of calendar/integrations are production-ready vs contract stubs?
- Can you modify `configs/config.yaml` to switch model family, working hours, and API/UI ports without touching code?
- Can you point to the tests that validate parser, prioritization, scheduling, memory, API, and workflow?